

5. Sample Source Code Implementation

DriverFactory.java

Purpose: Initialize and manage browser instances.

```
package utilities;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class DriverFactory {

    private static WebDriver driver;

    public static WebDriver getDriver() {

        if(driver == null) {
            driver = new ChromeDriver();
            driver.manage().window().maximize();
        }

        return driver;
    }

    public static void closeBrowser() {

        if(driver != null) {
            driver.quit();
        }
    }
}
```

LoginPage.java

Purpose: Handle Login Page Operations using Page Object Model.

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
```

```

import org.openqa.selenium.support.FindingBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {

    WebDriver driver;

    @FindingBy(id="email")
    WebElement username;

    @FindingBy(id="password")
    WebElement password;

    @FindingBy(id="loginBtn")
    WebElement loginButton;

    public LoginPage(WebDriver driver) {

        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    public void login(String user, String pass) {

        username.sendKeys(user);
        password.sendKeys(pass);
        loginButton.click();
    }
}

```

ProductPage.java

Purpose: Manage Product Operations.

```

package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindingBy;

public class ProductPage {

    WebDriver driver;

```

```

    @FindBy(id="searchBox")
    WebElement searchBox;

    @FindBy(id="searchBtn")
    WebElement searchButton;

    @FindBy(id="addCartBtn")
    WebElement addToCartButton;

    public ProductPage(WebDriver driver) {

        this.driver = driver;
    }

    public void searchProduct(String productName) {

        searchBox.sendKeys(productName);
        searchButton.click();
    }

    public void addProductToCart() {

        addToCartButton.click();
    }
}

```

CartPage.java

Purpose: Handle Shopping Cart Operations.

```

package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class CartPage {

    WebDriver driver;

    @FindBy(id="checkoutBtn")
    WebElement checkoutButton;

    public CartPage(WebDriver driver) {

```

```
        this.driver = driver;
    }

    public void proceedToCheckout() {

        checkoutButton.click();
    }
}
```

CheckoutPage.java

Purpose: Handle Checkout Process.

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class CheckoutPage {

    WebDriver driver;

    @FindBy(id="cardNumber")
    WebElement cardNumber;

    @FindBy(id="payBtn")
    WebElement payButton;

    public CheckoutPage(WebDriver driver) {

        this.driver = driver;
    }

    public void completePayment(String card) {

        cardNumber.sendKeys(card);
        payButton.click();
    }
}
```

LoginTest.java

Purpose: Validate Login Functionality.

```
package tests;

import org.junit.Assert;
import org.junit.Test;

import pages.LoginPage;
import utilities.DriverFactory;

public class LoginTest {

    @Test
    public void verifySuccessfulLogin() {

        LoginPage loginPage =
            new LoginPage(DriverFactory.getDriver());

        loginPage.login(
            "user@test.com",
            "Password123");

        Assert.assertTrue(true);
    }
}
```

CheckoutTest.java

Purpose: Validate Complete Purchase Flow.

```
package tests;

import org.junit.Test;

import pages.CartPage;
import pages.CheckoutPage;
import pages.ProductPage;

public class CheckoutTest {

    @Test
```

```
public void verifyCheckoutProcess() {  
  
    ProductPage product =  
        new ProductPage(driver);  
  
    product.searchProduct("Sports Shoe");  
    product.addProductToCart();  
  
    CartPage cart =  
        new CartPage(driver);  
  
    cart.proceedToCheckout();  
  
    CheckoutPage checkout =  
        new CheckoutPage(driver);  
  
    checkout.completePayment("1234567890123456");  
}  
}
```

Sample JUnit Execution Report

Test Suite Execution Summary

Total Tests Executed	: 25
Passed Tests	: 23
Failed Tests	: 2
Execution Time	: 5 Minutes
Pass Percentage	: 92%

Framework Design Principles Used

Page Object Model (POM)

Benefits:

- Improved code readability
- Reusable page methods
- Reduced code duplication
- Easier maintenance

Data Driven Testing

Benefits:

- Test multiple datasets
- Increase test coverage
- Reduce script duplication

Reusable Utilities

Benefits:

- Centralized browser management
- Easy configuration management
- Simplified reporting

Automation Coverage

The implemented automation framework covers:

- User Registration
- User Login
- Product Browsing
- Product Search
- Add To Cart
- Multiple Product Selection
- Checkout
- Order Generation
- Cart Clearance
- Order History Verification

This implementation demonstrates how Selenium WebDriver and JUnit can be used to automate end-to-end business workflows within the Sporty Shoes application.